

16: Transaction in kafka (Consumer)

Broker has the records, and below are the 3 possible use cases:

1. Transaction status : ONGOING

```
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: 0 lastSequence: 0 producerId: 0 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: true isControl: false deleteHo  
rizonMs: OptionalLong.empty position: 0 CreateTime: 1774875887784 size: 243 magic: 2 compressCodec: snappy crc: 1578994549 isValid: true  
| offset: 0 CreateTime: 1774875887784 keySize: 4 valueSize: 113 sequence: 0 headerKeys: [ __typeId__ ] key: 0-33 payload: ("orderId":"0-33","customerId":"CUST-2","productId"  
:"PROD-3","quantity":1,"totalAmount":99.99,"status":"CREATED")
```

There is no Control Record for it yet, so means its still in ONGOING state.

2. Transaction status : COMMIT

```
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: 0 lastSequence: 0 producerId: 0 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: true isControl: false deleteHo  
rizonMs: OptionalLong.empty position: 0 CreateTime: 1774875888100 size: 216 magic: 2 compressCodec: snappy crc: 2158642178 isValid: true  
| offset: 0 CreateTime: 1774875888100 keySize: 0 valueSize: 82 sequence: 0 headerKeys: [ __typeId__ ] key: PAY-0-33 payload: ("paymentId":"PAY-0-33","customerId":"CUST-2","t  
otalAmount":99.99,"status":"PAID")  
baseOffset: 1 lastOffset: 1 count: 1 baseSequence: -1 lastSequence: -1 producerId: 0 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: true isControl: true deleteH  
orizonMs: OptionalLong.empty position: 216 CreateTime: 1774875888151 size: 78 magic: 2 compressCodec: none crc: 3247449494 isValid: true  
| offset: 1 CreateTime: 1774875888151 keySize: 4 valueSize: 6 sequence: -1 headerKeys: [ ] endTxnMarker: COMMIT coordinatorEpoch: 0
```

3. Transaction status : ABORT

```
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: 0 lastSequence: 0 producerId: 0 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: true isControl: false deleteHo  
rizonMs: OptionalLong.empty position: 0 CreateTime: 1774901083813 size: 243 magic: 2 compressCodec: snappy crc: 172266883 isValid: true  
| offset: 0 CreateTime: 1774901083813 keySize: 4 valueSize: 113 sequence: 0 headerKeys: [ __typeId__ ] key: 0-33 payload: ("orderId":"0-33","customerId":"CUST-2","productId"  
:"PROD-3","quantity":1,"totalAmount":99.99,"status":"CREATED")  
baseOffset: 1 lastOffset: 1 count: 1 baseSequence: -1 lastSequence: -1 producerId: 0 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: true isControl: true deleteH  
orizonMs: OptionalLong.empty position: 243 CreateTime: 1774901084384 size: 78 magic: 2 compressCodec: none crc: 3188629820 isValid: true  
| offset: 1 CreateTime: 1774901084384 keySize: 4 valueSize: 6 sequence: -1 headerKeys: [ ] endTxnMarker: ABORT coordinatorEpoch: 0
```

Concept of LSO (Last Stable Offset):

LSO → offset of the first message that belongs to an OPEN (ongoing) Transaction.

A "read_committed" consumer CAN NOT read past the LSO. Even if there are committed or non transaction records.

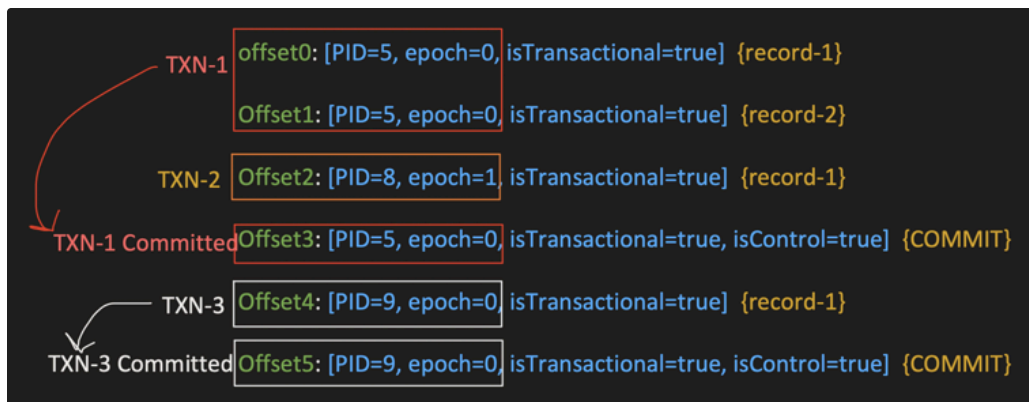
Example:

Topic: Order-events

Partition: P2

Logs:

Example:



LSO=2 (first record in an open transaction)

Maintained by broker and keep it in-memory.

Consumer

```
1 spring.kafka.consumer.properties.isolation.level=read_committed /
2 read_uncommitted (default)
```

In Consumer ,there are 2 parts :-

1. **read_uncommitted (default)**
2. **read_committed**

Now in **read_uncommitted (default):-**

Reads everything **except control records (isControl=true)**

- Ongoing transactional records
- Committed transactional records
- Aborted transactional records
- Non-transaction records

For above example:

Consumer will read:

- **Offset0** - part of committed txn-1
- **Offset1** - part of committed txn-1
- **Offset2** - Open (ongoing) txn-2
- **Offset4** - part of committed txn-3

Skips:

- **Offset3** - Control record for Txn-1
- **Offset5** - Control record for Txn-3

Now in **read_committed:-**

Reads:

- Committed transactional records
- Non-transactional records
- Skips Aborted transactional records
- Stops at LSO
- Skips control records(isControl = true) (Broker itself do not return it)

For above example:

Consumer will read:

- **Offset0** - part of committed txn
- **Offset1** - part of committed txn

LSO = 2, so consumer can not read it, as it denotes this Txn is still open.

Notice: even though Txn-3 is committed but consumer can not read it, because of LSO, this ensures that consumer never sees partially completed transactions preserving both ordering and atomicity guarantees.

How Consumer with "read_committed" skips ABORTED records?

This is interesting:

Aborted records are not filtered at Broker layer. Its filtered at Consumer layer.

But question is:

How consumer know, which offsets need to be skip ?

offset0: [PID=5, epoch=0, isTransactional=true] {record-1}

Offset1: [PID=5, epoch=0, isTransactional=true] {record-2}

Offset2: [PID=8, epoch=1, isTransactional=true] {record-1}

Offset3: [PID=5, epoch=0, isTransactional=true, isControl=true] {COMMIT}

Offset4: [PID=9, epoch=0, isTransactional=true] {record-1}

Offset5: [PID=9, epoch=0, isTransactional=true, isControl=true] {COMMIT}

Offset6: [PID=8, epoch=1, isTransactional=true, , isControl=true] {ABORT}

Offset7: [PID=8, epoch=1, isTransactional=true] {record-1} **(committed)**

Offset8: [PID=8, epoch=1, isTransactional=true, isControl=true] {COMMIT}

Here for "read_committed" consumer, broker will return:

- **offset0:** [PID=5, epoch=0, isTransactional=true] {record-1} **(committed)**
- **Offset1:** [PID=5, epoch=0, isTransactional=true] {record-2} **(committed)**

- **Offset2:** [PID=8, epoch=1, isTransactional=true] {record-1} **(Aborted)**
- **Offset4:** [PID=9, epoch=0, isTransactional=true] {record-1} **(committed)**
- **Offset7:** [PID=8, epoch=1, isTransactional=true] {record-1} **(committed)**

Skips Control records:

- **Offset3:** [PID=5, epoch=0, isTransactional=true, isControl=true] {COMMIT}
- **Offset5:** [PID=9, epoch=0, isTransactional=true, isControl=true] {COMMIT}
- **Offset6:** [PID=8, epoch=1, isTransactional=true, , isControl=true] {ABORT}
- **Offset8:** [PID=8, epoch=1, isTransactional=true, isControl=true] {COMMIT}

Now how consumer filter out this Aborted record?

Also Notice: there are 2 records with same PID and epoch

- **offset0:** [PID=5, epoch=0, isTransactional=true] {record-1} **(committed)**
- **Offset1:** [PID=5, epoch=0, isTransactional=true] {record-2} **(committed)**
- **Offset2:** [PID=8, epoch=1, isTransactional=true] {record-1} **(Aborted)**
- **Offset4:** [PID=9, epoch=0, isTransactional=true] {record-1} **(committed)**
- **Offset7:** [PID=8, epoch=1, isTransactional=true] {record-1} **(committed)**

So only Offset2 should be skipped, not Offset7.

Answer is provided by Broker itself:

For every transaction, which is ABORTED. Broker maintains a separate index file:

ex:

00000000000000000000.txnindex

```

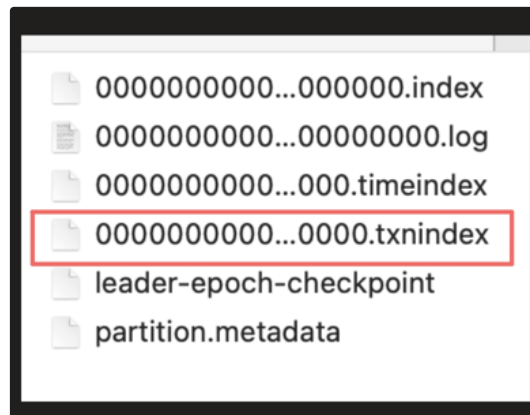
1  [
2      {
3          PID: 8,
4          Epoch: 1,
5          First offset Index: 2,
6          Control record offset (last offset): 6
7      }
8  ]

```

So, Consumer uses this file to know, what all records are aborted and skips them.

```
- offset0: [PID=5, epoch=0, isTransactional=true] {record-1} (committed) <----- Accepted, PID and epoch do not match ✓
- Offset1: [PID=5, epoch=0, isTransactional=true] {record-2} (committed) <----- Accepted, PID and epoch do not match ✓
- Offset2: [PID=8, epoch=1, isTransactional=true] {record-1} (Aborted) <----- Skipped, as PID=8, epoch=1 and offset 2 is within range [2-6] ✗
- Offset4: [PID=9, epoch=0, isTransactional=true] {record-1} (committed) <----- Accepted, PID and epoch do not match ✓
- Offset7: [PID=8, epoch=1, isTransactional=true] {record-1} (committed) <----- Accepted, as PID=8, epoch=1 but offset 7 is not in range [2-6] ✓
```

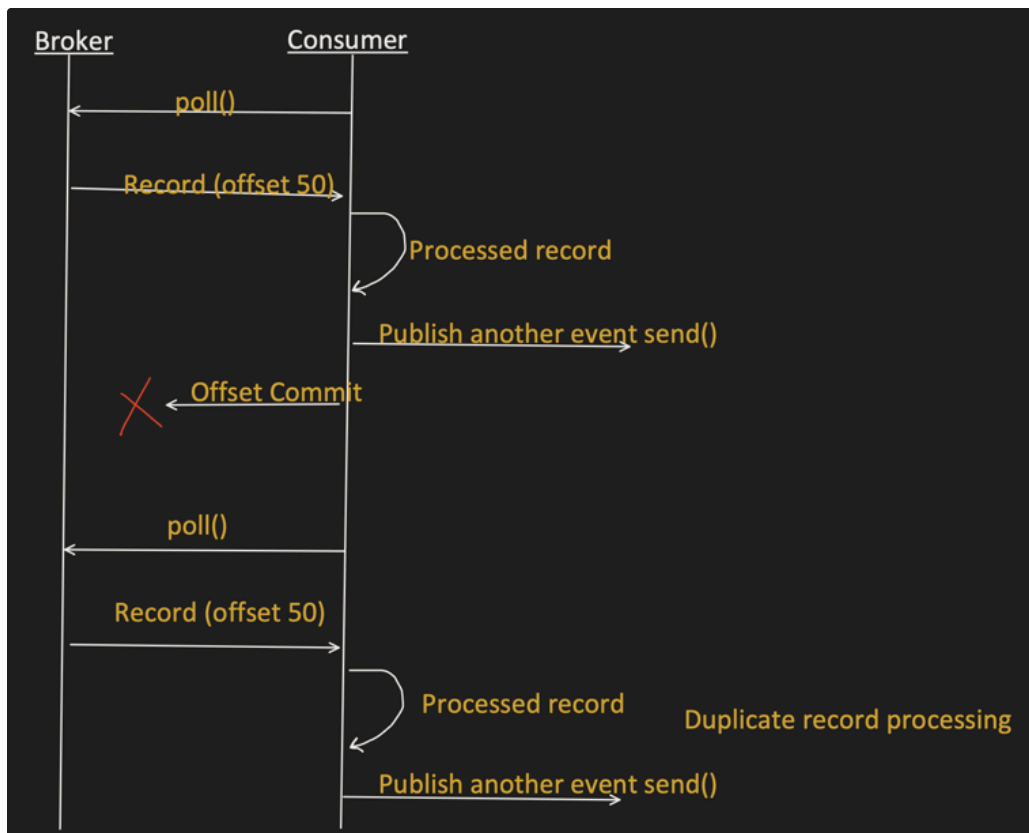
know



```
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka 2.13-4.2.0 % bin/kafka-dump-log.sh --deep-iteration --print-data-log \
--files /tmp/broker3-logs/order-events-0/00000000000000000000.txnindex
Dumping /tmp/broker3-logs/order-events-0/00000000000000000000.txnindex
version: 0 producerId: 0 firstOffset: 0 lastOffset: 1 lastStableOffset: 2
```

So this one problem we discussed exists at Consumer side.

Duplicate Processing (Read-Process-Write) - Consumer side problem



This is core **EOS (Exactly One Semantics)** problem:

```

1 | read -> process -> write -> commit offset
2 |

```

The above problem happens because: Write and Commit Offset is a separate step. So if any crash happens, could result in:

- Duplicate writes

That's where the name comes: **EOS (Exactly once)**, no duplicates.

So in this use-case :

Read → Process → Write → Commit offset

EOS achieved = read_committed Consumer + Idempotency + Transaction

In simple term, with the help of EOS, write and commit offset step is made atomic.

Read → Process → Write → Commit offset

Any failure, write and commit offset will not happen.

Implementation:

application.properties

```
1 .
2 . (other consumer properties)
3 .
4
5 spring.kafka.consumer.properties.isolation.level=read_committed
6 spring.kafka.consumer.enable-auto-commit=false
7
8
9 spring.kafka.producer.transaction-id-prefix=order-serv-
10 spring.kafka.producer.acks=all
11 spring.kafka.producer.enable-idempotence=true
12
```

```
1 @Component
2 public class OrderEventListener {
3
4     @Autowired
5     private KafkaTemplate<String, Object> kafkaTemplate;
6
7     @KafkaListener(topics = "order-events")
8     public void process(ConsumerRecord<String, Object> record) {
9
10         kafkaTemplate.executeInTransaction(ops -> {
11
12             // 1. READ happened and a record is received in this
13             method
14             String key = record.key();
15             Object value = record.value();
16
17             // 2. PROCESS, any business logic here if you want
18
19             // 3. WRITE (to another topic)
20             ops.send("payment-events", key, result);
21
22             // 4. SEND OFFSET TO TRANSACTION, we are just buffering
23             the offsets which need to be Committed when this txn is committed.
24             Map<TopicPartition, OffsetAndMetadata> offsets = new
25             HashMap<>();
26             offsets.put(
27                 new TopicPartition(record.topic(),
28                 record.partition()),
29                 new OffsetAndMetadata(record.offset() + 1)
30             );
31
```

```
29  
30         ops.sendOffsetsToTransaction(offsets, "order-group");  
31  
32         return null;  
33     });  
34 }  
35 }  
36
```